

UNIVERSITY OF ENGINEERING & TECHNOLOGY

PESHAWAR

Computer Programming

LABORATORY REPORT

Lab No. 06

Topic:

The for Loop in C++

Submitted By:

Muhammad Adil

Registration No: 0669

Semester: Spring 2026

Table of Contents

1. Objectives

2. Theory

2.1 The for Loop

2.2 Flow Control in a for Loop

2.3 Nested for Loop

3. Lab Tasks

Task 1: Numbers 1-10 using for Loop

Task 2: Multiplication Table using Nested for Loop

Task 3: Sum using for Loop

Task 4: Triangle Pattern using Nested for Loop

4. Example Programs

Example 1: Print Values 5 to 1

Example 2: Prime Number Check

Example 3: Number Pattern

5. Outputs and Results

6. Conclusion

1. Objectives

The objectives of this laboratory session are to learn and apply the following C++ loop constructs:

- Understand and implement the **for loop** for repetitive execution with known iteration counts.
- Understand and implement **nested for loops** for multi-dimensional iteration and pattern printing.
- Apply for loop constructs to solve practical programming problems including pattern generation and mathematical computations.

2. Theory

2.1 The for Loop

A **for loop** is a repetition control structure that allows you to efficiently write a loop that needs to execute a specific number of times. The for loop is used when a value runs from a starting point to an ending point with a constant increment or decrement.

The general syntax of the for loop is:

```
for (initialization; condition; update)
{
    // statements to execute
}
```

The three components of the for loop header are:

- **Initialization:** Executed once before the loop begins. Used to declare and initialize the loop control variable.
- **Condition:** Evaluated before each iteration. If true, the loop body executes; if false, the loop terminates.
- **Update:** Executed after each iteration. Used to increment or decrement the loop control variable.

2.2 Flow Control in a for Loop

The flow of execution in a for loop follows these steps:

1. The **initialization** of the loop variable is carried out first, and only once. This step allows you to declare and initialize any loop control variables.
2. Next, the **condition** is evaluated. If it is true, the body of the loop is executed. If it is false, the body of the loop does not execute and the flow of control jumps to the next statement after the for loop.
3. After the body executes, the flow of control jumps back up to **update the variable**. This statement allows you to update the loop control variables.
4. The **condition is evaluated again**. If it is true, the loop executes and the process repeats itself. After the condition becomes false, the for loop terminates.

2.3 Nested for Loop

A **nested for loop** is a for loop placed inside another for loop. For each iteration of the outer loop, the inner loop completes all of its iterations. This construct is particularly useful for working with matrices, generating patterns, and multi-dimensional data processing.

The general syntax is:

```
for (initialization_outer; condition_outer; update_outer)
{
    for (initialization_inner; condition_inner; update_inner)
    {
        // inner loop statements
    }
    // outer loop statements
}
```

3. Lab Tasks

Task 1: Numbers 1-10 using for Loop

Objective: Write a C++ program that uses a for loop to display numbers from 1 to 10. Display each number on a separate line.

Program Code

```
#include <iostream>
using namespace std;

int main()
{
    cout << "Numbers from 1 to 10 using for loop:" << endl;
    cout << "======" << endl;

    for (int i = 1; i <= 10; i++)
    {
        cout << i << endl;
    }

    return 0;
}
```

Explanation

The program initializes the loop variable `i` to 1. The for loop continues as long as `i <= 10`. After each iteration, `i` is incremented by 1 using the post-increment operator `i++`. The loop prints each value of `i` on a separate line, producing the output sequence from 1 to 10.

Task 2: Multiplication Table using Nested for Loop

Objective: Write a C++ program that generates a multiplication table from 1 to 10 using nested for loops. Display the table in a formatted manner.

Program Code

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    cout << "Multiplication Table (1 to 10):" << endl;
    cout << "===== " << endl << endl;

    // Print header row
    cout << setw(4) << " ";
    for (int h = 1; h <= 10; h++)
    {
        cout << setw(4) << h;
    }
    cout << endl;

    // Print separator
    cout << " ----- " << endl;

    // Print table rows using nested loops
    for (int row = 1; row <= 10; row++)
    {
        cout << setw(3) << row << "|";
        for (int col = 1; col <= 10; col++)
        {
            cout << setw(4) << row * col;
        }
        cout << endl;
    }

    return 0;
}
```

Explanation

The program uses nested for loops to generate a 10x10 multiplication table. The outer loop iterates through rows (1 to 10), and for each row, the inner loop iterates through columns (1 to 10), printing the product of the current row and column. The `setw()` manipulator from `<iomanip>` ensures consistent column widths for proper alignment.

Task 3: Sum using for Loop

Objective: Write a C++ program that prompts the user to enter a positive integer. Use a for loop to calculate the sum of all numbers from 1 to the entered integer. Display the calculated sum.

Program Code

```
#include <iostream>
using namespace std;

int main()
{
    int n;
    int sum = 0;

    cout << "Enter a positive integer: ";
    cin >> n;

    // Input validation
    if (n <= 0)
    {
        cout << "Please enter a positive integer." << endl;
        return 1;
    }

    // Calculate sum using for loop
    for (int i = 1; i <= n; i++)
    {
        sum = sum + i;
    }

    cout << "The sum of numbers from 1 to " << n
         << " is: " << sum << endl;

    return 0;
}
```

Explanation

The program prompts the user for a positive integer `n`. The for loop initializes `i` to 1 and iterates until `i` exceeds `n`, adding each value of `i` to the accumulator variable `sum`. The program includes input validation to ensure only positive integers are processed. Note: The sum can also be computed using the formula $n(n+1)/2$ for an $O(1)$ solution.

Task 4: Triangle Pattern using Nested for Loop

Objective: Write a C++ program that uses nested for loops to print a pattern of asterisks forming a triangle shape. The number of rows should be determined by user input.

Program Code

```
#include <iostream>
using namespace std;

int main()
{
    int rows;

    cout << "Enter the number of rows for the triangle: ";
    cin >> rows;

    if (rows <= 0)
    {
        cout << "Please enter a positive integer." << endl;
        return 1;
    }

    cout << endl;

    // Outer loop controls the number of rows
    for (int i = 1; i <= rows; i++)
    {
        // Inner loop prints asterisks for each row
        for (int j = 1; j <= i; j++)
        {
            cout << "* ";
        }
        cout << endl;
    }

    return 0;
}
```

Explanation

The program uses nested for loops to create a right-angled triangle pattern. The outer loop controls the row number (from 1 to the user-specified count). For each row `i`, the inner loop prints `i` asterisks. As the row number increases, the number of asterisks printed per row increases, forming a triangular pattern.

4. Example Programs

Example 1: Print Values 5 to 1

This program demonstrates a for loop with decrement to print values from 5 down to 1.

Program Code

```
#include <iostream>
using namespace std;

int main()
{
    for (int i = 5; i > 0; i--)
    {
        cout << i << endl;
    }
    cout << "Outside the loop now" << endl;
    return 0;
}
```

Output

Sample Output:

```
5
4
3
2
1
Outside the loop now
```

Example 2: Prime Number Check

This program uses a for loop to check whether a given number is prime or not.

Program Code

```
#include <iostream>
using namespace std;

int main()
{
    int num;
    cout << "Enter a number please \n";
    cin >> num;

    int flag = 0;
    for (int i = 2; i < num; i++)
    {
        if (num % i == 0)
        {
            flag = 1;
            break;
        }
    }

    if (flag == 1)
    {
        cout << num << " is not a prime number \n";
    }
    else
    {
        cout << num << " is a prime number \n";
    }

    return 0;
}
```

Explanation

The program checks if a number is prime by testing divisibility from 2 up to `num - 1`. If any divisor is found, the `flag` is set to 1 and the loop exits early using `break`. If no divisor is found after all iterations, the number is prime. This is a straightforward approach; more optimized algorithms check divisors only up to the square root of the number.

Example 3: Number Pattern

This program uses nested for loops to print a decreasing number pattern.

Program Code

```
#include <iostream>
using namespace std;

int main()
{
    for (int num = 7; num > 0; num--)
    {
        for (int j = 1; j <= num; j++) // nested loop
        {
            cout << j << " ";
        }
        cout << "\n";
    }
    return 0;
}
```

Output

Sample Output:

```
1 2 3 4 5 6 7
1 2 3 4 5 6
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

Explanation

The outer loop starts with `num = 7` and decrements down to 1. For each value of `num`, the inner loop prints numbers from 1 to `num`. As `num` decreases with each outer iteration, each subsequent row contains fewer numbers, creating a triangular pattern that decreases in width.

5. Outputs and Results

Task 1 Output

Program Output:

```
Numbers from 1 to 10 using for loop:
=====
1
2
3
4
5
6
7
8
9
10
```

Task 2 Output

Program Output:

```
Multiplication Table (1 to 10):
=====
      1   2   3   4   5   6   7   8   9  10
-----
1|   1   2   3   4   5   6   7   8   9  10
2|   2   4   6   8  10  12  14  16  18  20
3|   3   6   9  12  15  18  21  24  27  30
... (continues up to row 10)
```

Task 3 Output

Program Output:

```
Enter a positive integer: 10
The sum of numbers from 1 to 10 is: 55

Enter a positive integer: 5
The sum of numbers from 1 to 5 is: 15

Enter a positive integer: 100
The sum of numbers from 1 to 100 is: 5050
```

Task 4 Output

Program Output:

Enter the number of rows for the triangle: 5

```
*
* *
* * *
* * * *
* * * * *
```

6. Conclusion

In this laboratory session, we successfully studied and implemented the **for loop** and **nested for loop** constructs in C++. These loop structures provide efficient mechanisms for executing code blocks a specific number of times, making them essential tools for repetitive tasks in programming.

Through the lab tasks and example programs, we demonstrated the following key concepts:

- Using the for loop with increment to display sequential numbers (Task 1).
- Using nested for loops with formatted output to generate a multiplication table (Task 2).
- Using the for loop with user input to calculate the sum of natural numbers (Task 3).
- Using nested for loops to generate geometric patterns (Task 4).
- Using for loops with decrement for reverse-order iteration (Example 1).
- Combining for loops with conditional statements and the `break` keyword for prime number checking (Example 2).

The for loop is particularly useful when the number of iterations is known in advance, as it consolidates initialization, condition, and update into a single line. The nested for loop extends this capability to multi-dimensional problems, enabling solutions for pattern printing, matrix operations, and table generation.

Understanding these loop structures is fundamental for advanced programming concepts including array manipulation, string processing, file I/O operations, and algorithm implementation. Mastery of loop constructs enables programmers to write efficient, concise, and maintainable code for a wide range of computational problems.